S2D-06: ArrayMovingSum for Extended Timeframes

> **Series:** S2D — Splunk to Dynatrace Migration | **Notebook:** 6 of 9 | **Created:** January 2026 | **Last Updated:** 11/02/2026

Overview

Anomaly Detectors have a maximum sliding window of 60 minutes. For alerts that need to evaluate longer timeframes while still using continuous monitoring, the `arrayMovingSum` function provides a solution.

This notebook explains how to use `arrayMovingSum` to create rolling aggregations that can be used with Anomaly Detectors.

Table of Contents

1. The Problem
2. How ArrayMovingSum Works
3. Basic Example
4. Limitations
5. Using with Anomaly Detectors
6. Dashboard/Report Example (Longer Timeframes)
7. Other Array Functions
8. When ArrayMovingSum Isn't Enough

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled
Permissions	`storage:logs:read`, `storage:buckets:read`
Knowledge	Basic DQL and array functions

Learning Objectives

By the end of this notebook, you will be able to:

1. Understand the `arrayMovingSum` function
2. Create rolling sum queries for extended timeframes
3. Apply this pattern to Anomaly Detectors
4. Recognize limitations and alternatives

The Problem

Many Splunk alerts evaluate over timeframes longer than 1 hour:

- "Alert if more than 500 errors in the last 4 hours"
- "Alert if average response time exceeds 2s over the past 2 hours"

Anomaly Detectors have a **maximum sliding window of 60 minutes**.

The Solution: ArrayMovingSum

The `arrayMovingSum` function calculates a rolling sum across timeseries data points, allowing you to aggregate values over longer periods while maintaining 1-minute granularity.

How ArrayMovingSum Works

```
arrayMovingSum(array, window)
```

Parameter	Description
`array`	The timeseries array to aggregate
`window`	Number of data points to sum

Timeframe Calculation

The effective timeframe depends on:
- **Interval**: The timeseries interval (e.g., 1m)
- **Window**: The number of points to sum

```
Effective Timeframe = Interval × Window
```

Example: `interval:1m` with `window:60` = 60-minute rolling sum

Basic Example

Create a 60-minute rolling sum of error logs:

```
// 60-minute rolling sum of error logs
fetch logs, from:-24h
| filter status == "ERROR"
| makeTimeseries error_count = count(), interval:1m
| fieldsAdd error_count_last_1h = arrayMovingSum(error_count, 60)
| fieldsRemove error_count
```

Understanding the Output

Each data point in `error_count_last_1h` represents the **sum of errors in the preceding 60 minutes**.

Time	error_count (per minute)	error_count_last_1h (rolling 60m)
10:00	5	120 (sum of 9:01-10:00)
10:01	3	118 (sum of 9:02-10:01)
10:02	8	123 (sum of 9:03-10:02)

Limitations

Maximum Window Size: 60

The `arrayMovingSum` window parameter has a **maximum value of 60**.

With a 1-minute interval, this means:
- **Maximum effective timeframe: 60 minutes**

Increasing the Interval?

You could increase the interval to extend the timeframe:

Interval	Window	Effective Timeframe
1m	60	60 minutes
5m	60	300 minutes (5 hours)
10m	60	600 minutes (10 hours)

Intervals snap to supported values. `makeTimeseries` silently coarsens an interval it does not support: `interval:4m` returns 5-minute points, and `interval:1m` over 7 days returns 10-minute points, with no warning. Read the returned `interval` field before you multiply it by the window.

However: Anomaly Detectors require a **1-minute interval**, so this workaround only applies to dashboards and reports, not continuous alerting.

Using with Anomaly Detectors

For Anomaly Detectors, you must use `interval:1m` and `window:60`.

Custom-alert query (no `from:` / `to:`, per S2D-04 Step 1):

```
// Paste into the custom alert WITHOUT from:/to:
// dt.entity.cloud_application is deprecated (semantic dictionary); its
Smartscape successors are not on log records yet (10/2026)
fetch logs
| filter status == "ERROR"
| filter matchesPhrase(k8s.deployment.name, "checkout-service")
| makeTimeseries error_count = count(), by:{dt.entity.cloud_application},
interval:1m
| fieldsAdd error_count_1h = arrayMovingSum(error_count, 60)
| fieldsRemove error_count
```

Notebook preview of the same query:

```
// Notebook PREVIEW of the custom-alert query above (60-minute rolling sum)
// Remove from:-24h when you paste it into the custom alert
// dt.entity.cloud_application is deprecated (semantic dictionary); its
Smartscape successors are not on log records yet (10/2026)
fetch logs, from:-24h
| filter status == "ERROR"
| filter matchesPhrase(k8s.deployment.name, "checkout-service")
| makeTimeseries error_count = count(), by:{dt.entity.cloud_application},
interval:1m
| fieldsAdd error_count_1h = arrayMovingSum(error_count, 60)
| fieldsRemove error_count
```

Configuring the Detector

When creating the Anomaly Detector:

Setting	Value	Rationale
Threshold	Total errors in 60 minutes	e.g., 100
Sliding Window	60 minutes (the maximum)	The query only sees the data its sliding window fetches (see below)
Violating Samples	1	A sample above the threshold already means that many errors fell inside 60 minutes

> **Why not a 1-minute sliding window.** The configuration page says the sliding window *"defines how many one-minute samples must violate the threshold"*, and the

> DQL writing guide says *"The custom alert retrieves data from Grail based on predefined sliding windows."* `arrayMovingSum` can only add up the points the query returns. With a 1-minute window the query gets about one minute of data, so each "60-minute" sample is really a 1-minute count compared against a 60-minute threshold, and the alert would rarely fire.
>
> **The first 59 points are partial sums.** In any query window, point *n* of `arrayMovingSum(…, 60)` adds up only the points up to it. Measured on a validation tenant on 10/14/2026: over `from:-60m` the series had 61 points, the first was 1,868 (one minute of errors) and the last 116,116 (the 60-minute sum); over `from:-1m` the "rolling sum" was 966, a single minute. Preview the query above with `from:-60m` to see what one evaluation receives.
>
> **Not tested on a live custom alert.** These settings are derived from the two quoted pages and the measurements, not from a configured detector. Confirm them on a non-production custom alert before relying on them. A 60-minute sliding window is also the most a detector can evaluate, so this recipe cannot cover a longer lookback; for that, see S2D-05.
>
> **Sources:** Anomaly detection configuration (DT docs), Anomaly Detection DQL writing guide (DT docs). **Derived:** the 60-minute sliding window combines the sliding-window definition and the data-retrieval sentence with the measured partial sums.

Dashboard/Report Example (Longer Timeframes)

For dashboards and reports (not Anomaly Detectors), you can use longer intervals:

```
// 4-hour rolling sum for dashboard visualization
// 5-minute interval x 48 points = 240 minutes (4 hours)
// Intervals snap to supported values - check the returned `interval` field
fetch logs, from:now()-24h
| filter status == "ERROR"
| makeTimeseries error_count = count(), interval:5m
| fieldsAdd error_count_4h = arrayMovingSum(error_count, 48)
| fieldsRemove error_count
```

Other Array Functions

Related functions for rolling calculations:

Function	Description
`arrayMovingSum`	Rolling sum
`arrayMovingAvg`	Rolling average
`arrayMovingMin`	Rolling minimum
`arrayMovingMax`	Rolling maximum

```
// Rolling average response time over 30 minutes
// response_time is not a built-in field: parse it from the content first
fetch logs, from:-24h
| parse content, "DATA? 'response_time=' DOUBLE:response_time"
| filter isNotNull(response_time)
| makeTimeseries avg_response = avg(response_time), interval:1m
| fieldsAdd avg_response_30m = arrayMovingAvg(avg_response, 30)
```

When ArrayMovingSum Isn't Enough

If you need timeframes exceeding 60 minutes with Anomaly Detectors, you have these options:

Approach	Use Case	Trade-offs
Workflow-based alerting	Any timeframe	No Dynatrace Intelligence, consumes workflow hours
Metric extraction	High-volume queries	Requires OpenPipeline config
Reduce threshold	Proportional reduction	May increase false positives

Proportional Threshold Reduction Example

If Splunk alert is: 500 errors in 4 hours

Convert to 1-hour equivalent:
- 500 ÷ 4 = 125 errors per hour
- Set Dynatrace Intelligence threshold to 125 with arrayMovingSum over 60 minutes

Caveat: This assumes errors are evenly distributed, which may not match actual patterns.

Summary

Scenario	Solution
Alert over ≤60 minutes	Standard Anomaly Detector
Alert over exactly 60 minutes	ArrayMovingSum with window:60
Alert over >60 minutes	Use Workflows (see S2D-05)
Dashboard visualization >60 minutes	ArrayMovingSum with larger interval

Next Steps

- **S2D-07: Metric Creation** - For performance-critical alerting queries
- **S2D-05: Workflow Alerts** - For timeframes exceeding 60 minutes

References

- ArrayMovingSum Documentation
- Array functions (DT docs)
- anomaly-detection (DT docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.